BÖLÜM 2 / MODULE 2

Kubernetes ile Tanışma – Orkestra Şefi

Introduction to Kubernetes – The Conductor

Neden sadece Docker yetmez? K8s mimarisi, Control Plane, Worker Node'lar, Pod kavramı ve ilk Spring Boot manifestomuzu çalıştırma rehberi.

Why Docker alone falls short: K8s architecture, Control Plane, Worker Nodes, Pods, declarative models, and running your first Spring Boot manifest.

7

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Neden Sadece Docker Yetmez? Kubernetes Neyi Çözer?

Why Docker Alone Isn't Enough: What Kubernetes Actually Solves

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Elinizde tek bir usta ve 1 fırın varken Docker harikadır. Ama ya restoranınız bir anda 10.000 şubeli bir dünya zincirine dönüşürse? Gece saat 03:00'te bir sunucunun fişi çekildiğinde, içindeki 50 konteyneri kim diğer sunuculara taşıyacak? Trafik 10 katına çıktığında kim anında 40 yeni konteyner açacak? İşte Kubernetes, 1.000 gemiyi ve 100.000 konteyneri tek bir parmak şıklatmasıyla yöneten baş amiraldir!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Docker is great when managing a single coffee stall. But what happens when you run a global chain of 10,000 shops? If a server catches fire at 3 AM, who wakes up to resurrect the 50 crashed containers on another server? Who adds 40 extra containers when traffic surges 1000%? Kubernetes is the supreme admiral orchestrating a fleet of thousands of ships automatically!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Docker tek bir host (tekil sunucu) üzerinde konteyner çalıştırır. Kubernetes (K8s) ise onlarca hatta binlerce fiziksel sunucuyu tek bir devasa soyut süper-bilgisayar gibi birleştirir. Temel yetenekleri: 1) Automated Scheduling (Konteyneri en uygun kaynaklı sunucuya yerleştirme), 2) Self-Healing (Çöken konteynerleri yeniden başlatma, yanıt vermeyen düğümleri tahliye etme), 3) Horizontal Auto-Scaling (CPU/Trafiğe göre otomatik çoğalma), 4) Service Discovery & Load Balancing (Trafiği sağlıklı konteynerlere eşit dağıtma).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Docker manages containers on a single host. Kubernetes abstracts an entire fleet of physical/virtual machines into a single unified elastic compute fabric. Its core primitives deliver: 1) Automated scheduling & bin packing, 2) Self-healing & node drain automation, 3) Horizontal Pod Autoscaling (HPA), and 4) Automated discovery and Layer 4/7 load balancing.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Docker ile manuel çile (Sunucu çökerse bittiniz!):
ssh server-01 "docker run -d -p 8080:8080 my-coffee-app:1.0"

# Kubernetes ile bildirici güç (Sunucu patlasa da K8s başka sunucuda anında açar):
kubectl apply -f deployment.yaml
```

⚠️ Acemi Tuzağı & Pro İpucu

🇹🇷 AMAN DİKKAT

Kubernetes bir sihirli değnek değildir! Kötü yazılmış, bellek sızdıran veya 'graceful shutdown' desteklemeyen bir Spring Boot uygulamasını K8s'e koyarsanız sadece hatalarınızı daha hızlı çoğaltmış olursunuz.

⚠️ Gotchas & Pro-Tips

🇬🇧 WATCH OUT

Kubernetes does not fix broken code. If your Spring Boot app leaks memory or crashes on SIGTERM, K8s will just reboot and crash it faster.

🎯 Özet Çıkarımlar

- Docker konteynerleri çalıştırır; Kubernetes binlerce konteyneri orkestre eder.
- Kendi kendini iyileştirme (Self-Healing) sayesinde gece mühendis uykusundan uyanmaz.
- Yüzlerce sunucu tek bir dev işlemci ve bellek havuzuna dönüştürülür.

🎯 Key Takeaways

- Docker runs containers; Kubernetes orchestrates thousands across clusters.
- Self-healing keeps systems alive without manual midnight engineering interventions.
- Aggregates disparate physical nodes into a singular logical compute pool.

8

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

K8s Mimarisi: Control Plane (Beyin) vs Worker Nodes (Kas Gücü)

K8s Architecture: Control Plane (Brain) vs Worker Nodes (Muscle)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Dev bir inşaat şirketi düşünün. Şirketin lüks genel merkezinde (Control Plane) oturan yöneticiler vardır: 1) Resepsiyonist (API Server): Tüm emirleri kabul eder. 2) Noter (etcd): Yapılan her sözleşmeyi kasaya kilitler. 3) Teşrifatçı (Scheduler): Hangi işçinin hangi şantiyeye gideceğini seçer. 4) Denetmen (Controller Manager): Eksik işçi var mı diye kontrol eder. Şantiyede (Worker Node) ise baretini takıp harç karan işçiler (Kubelet ve Kube-Proxy) bulunur!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Picture a massive global construction firm. The air-conditioned headquarters is the Control Plane: 1) Receptionist (API Server): The only entry door for orders. 2) Master Vault (etcd): Immutable ledger storing all facts. 3) Dispatcher (Scheduler): Decides which worker goes to which site. 4) Inspector (Controller Manager): Constantly counts headcount. The actual construction sites are Worker Nodes with Kubelet (Site Foreman) and Kube-Proxy (Traffic Controller)!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Control Plane bileşenleri: 1) kube-apiserver (REST API sunar; tüm bileşenler sadece onunla konuşur), 2) etcd (Raft konsensüs algoritması kullanan dağıtık anahtar-değer veritabanı; kümenin tek gerçek kaynağı), 3) kube-scheduler (Kaynak ihtiyaçlarına göre Pod'u en uygun Node'a atar), 4) kube-controller-manager (Deployment, Node, Endpoints kontrol döngülerini çalıştırır). Worker Node bileşenleri: 1) kubelet (Düğümdeki K8s ajanı; konteynerleri başlatır ve denetler), 2) kube-proxy (Ağ kurallarını ve iptables/IPVS yönlendirmelerini yönetir), 3) Container Runtime (containerd/CRI-O).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Control Plane internals: 1) kube-apiserver (central REST gateway; all cluster state passes here), 2) etcd (high-availability Raft-backed key-value store), 3) kube-scheduler (assigns unscheduled pods to optimal nodes), 4) kube-controller-manager (executes reconciliation loops). Worker Node components: 1) kubelet (node agent interacting with CRI to manage containers), 2) kube-proxy (maintains network packet filtering rules via iptables/IPVS), 3) Container Runtime (CRI-O / containerd).

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Control plane ve worker node'ların durumunu sorgula:
kubectl get nodes -o wide

# Cluster bileşenlerinin sağlık durumunu kontrol et:
kubectl get componentstatuses 2>/dev/null || kubectl get --raw='/readyz'
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

etcd veritabanı çökerse tüm küme kör olur! Üretim ortamında etcd her zaman tek sayılı (3 veya 5 düğüm) quorum mimarisiyle ve yüksek hızlı SSD disklerle çalıştırılmalıdır.

🇬🇧 WATCH OUT

If etcd loses quorum, your entire cluster enters read-only paralysis! Always run 3 or 5 odd-numbered etcd nodes on dedicated NVMe storage.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- API Server kümenin tek giriş kapısıdır; kimse doğrudan etcd ile konuşamaz.
- Kubelet, API Server'dan aldığı talimatları yerel konteyner motoruna iletir.
- Düğümler arızalansa bile Control Plane pod'ları başka düğümlere taşır.

- The API Server is the sole gateway; components never talk directly to etcd.
- Kubelet ensures containers described in PodSpecs are running and healthy.
- When a worker node dies, the Control Plane reschedules pods seamlessly.

9

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

En Temel Atom: Pod Nedir? Neden Doğrudan Konteyner Değil?

The Atomic Unit: What is a Pod and Why Not Bare Containers?

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

K8s dünyasında tek başına konteyner kiralayamazsınız; en küçük kiralık daire 'Pod'dur! Pod bir bezelye kabuğu, konteynerler ise içindeki bezelyelerdir. Çoğunlukla bir pod'da 1 bezelye (örneğin Spring Boot) yaşar. Ama bazen ana uygulamanın yanına ona yardımcı olan bir oda arkadaşı (Sidecar Konteyneri: örneğin log toplayıcı veya güvenlik ajanı) koyarsınız. Aynı pod'daki konteynerler aynı banyoyu (IP adresi) ve aynı buzdolabını (ortak disk) paylaşır!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

In Kubernetes, you cannot rent a bare container; the smallest rentable apartment is a 'Pod'! A Pod is a pea pod, and containers are the peas inside. Usually, a pod hosts 1 main pea (your Spring Boot microservice). But sometimes, you pair it with a helpful roommate (Sidecar container: log shipper, envoy proxy). All containers in the same Pod share the exact same apartment IP and localhost loopback!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod, bir veya daha fazla konteynerin aynı Linux Network Namespace'ini, IPC namespace'ini ve depolama birimlerini paylaştığı soyut bir yapıdır. Bir pod ayağa kalktığında K8s önce 'pause' adında minik bir altyapı konteyneri başlatır. Bu pause konteyneri IP adresini ve ağ arayüzünü alır. Ardından gelen Spring Boot konteyneri ve sidecar konteynerleri bu ağ alanına katılır (`--net=container:pause`). Bu yüzden aynı pod içindeki iki farklı konteyner birbiriyle `localhost:port` üzerinden sıfır ağ gecikmesiyle konuşabilir.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

A Pod encapsulates one or more co-located containers sharing Linux IPC, UTS, and Network namespaces via a lightweight 'pause' container. When a Pod initializes, the pause container claims the Pod IP. Subsequent app and sidecar containers attach to this namespace, allowing them to communicate over `localhost` with zero network serialization overhead and shared volumes.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Tek Pod içinde 2 konteyner örneği (Spring Boot + Log Tailer Sidecar):
apiVersion: v1
kind: Pod
metadata:
  name: coffee-with-sidecar
spec:
  containers:
    - name: spring-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
    - name: log-shipper-sidecar
      image: alpine:latest
      command: ["sh", "-c", "while true; do echo 'Sidecar logging...'; sleep 10; done"]
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

İki bağımsız iş yapan mikroservisi asla aynı pod'un içine koymayın! Pod'lar birlikte ölür ve birlikte ölçeklenir. Sadece birbirinden ayrılamaz derecede kenetlenmiş yardımcı araçlar (proxy, log forwarder) sidecar yapılmalıdır.

🇬🇧 WATCH OUT

Never combine two independent microservices inside the same Pod! Containers in a pod scale together. Use sidecars strictly for tightly coupled helpers.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Pod, Kubernetes'in zamanladığı ve yönettiği en küçük konuşlandırma birimidir.
- Aynı Pod içindeki konteynerler localhost üzerinden haberleşir ve IP paylaşır.
- Çoğu senaryoda 1 Pod = 1 Uygulama Konteyneri kuralı geçerlidir.

- A Pod is the smallest atomic unit of scheduling in Kubernetes.
- Containers inside the same Pod share an IP address and reach each other on localhost.
- The standard pattern is 1 App Container per Pod, unless utilizing a Sidecar.

10

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Bildirici (Declarative) Model vs Emirsel (Imperative) Model

Declarative Model vs Imperative Model: Desired State Philosophy

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Emirsel Model (Imperative): Garsona 'Mutfaktan bardağı al, 3 adım yürü, musluğu aç, suyu doldur, bana getir' demek gibidir. Bir adım aksarsa her şey kilitlenir. Bildirici Model (Declarative) ise şudur: 'Masanın üzerinde HER ZAMAN 3 bardak su istiyorum!' Biri bardağı kırarsa, garson hiç sormadan anında yeni bir bardak su koyar. K8s bildirici modelle çalışır: İstenen Durum (Desired State) = Gerçek Durum (Actual State)!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Imperative Model: Shouting micromanaged commands: 'Walk 5 steps, grab glass, fill water, bring it'. If step 3 fails, the system crashes. Declarative Model: 'I want 3 glasses of water on this table AT ALL TIMES.' If an earthquake shatters a glass, the waiter immediately replaces it without being asked. That is Kubernetes: Desired State equals Actual State!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kubernetes'in kalbinde 'Reconciliation Loop' (Mutabakat Döngüsü) yatar. Siz bir YAML manifestosu verdiğinizde bu 'Desired State' (İstenen Durum) olarak etcd'ye yazılır. İlgili kontrolcü (örneğin DeploymentController) durmaksızın şu döngüyü çalıştırır: 1) Observe (Mevcut durumu oku: kaç pod çalışıyor?), 2) Diff (İstenen ile mevcut arasındaki farkı bul: 3 istenmiş, 2 var!), 3) Act (Farkı kapat: 1 yeni pod oluştur). Bu sayede dışarıdan bir pod manuel silinse bile K8s saniyeler içinde farkı onarır.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kubernetes operates via an infinite 'Reconciliation Loop'. You declare the Desired State in YAML, stored in etcd. Controllers execute a continuous 3-step loop: 1) Observe actual cluster state, 2) Diff actual against desired state, 3) Actuate corrective API calls to converge actual state back to desired state. Self-healing is an inherent byproduct of this architecture.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```
# Emirsel Komut (Tavsiye Edilmez - İz bırakmaz):
kubectl run my-coffee-pod --image=ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0

# Bildirici Manifest (Kurumsal Standart - Git'e konabilir):
# coffee-deployment.yaml dosyasını uygula:
kubectl apply -f coffee-deployment.yaml

# K8s istenen durum ile mevcut durum arasındaki farkı anında kapatır!
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Canlı kümede doğrudan `kubectl edit` veya emirsel komutlarla değişiklik yaparsanız, bir sonraki GitOps veya `kubectl apply` çalışmasında bu manuel değişiklikler ezilir ve kaybolur!

🇬🇧 WATCH OUT

Making ad-hoc imperative edits directly in production leads to configuration drift; the next automated pipeline run will overwrite your changes.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Bildirici modelde 'nasıl yapılacağını' değil, 'neyin olması gerektiğini' söyleriz.
- Reconciliation döngüsü 7/24 istenen durum ile gerçek durumu eşitler.
- Tüm altyapı YAML dosyaları olarak Git'te versiyonlanabilir (Altyapı Kod Olarak - IaC).

- In declarative systems, you describe WHAT the state should be, not HOW to get there.
- The reconciliation loop runs 24/7, continuously eliminating drift.
- Declarative YAML manifests enable version-controlled Infrastructure as Code (IaC).

11

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

Yerel K8s Laboratuvarı: Minikube / Kind ve kubectl Temelleri

Local K8s Lab: Minikube / Kind and kubectl Command Mastery

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Koca bir Boeing 747 uçağını uçurmayı öğrenmek için hemen gerçek yolcu uçağına binmezsiniz; uçuş simülatöründe antrenman yaparsınız! Minikube veya Kind (Kubernetes IN Docker), laptopunuzun içinde güvenle çarpıp parçalayabileceğiniz minyatür bir Kubernetes kümesidir. `kubectl` ise bu uçağın direksiyonu ve vites koludur.

💡 ELI5 Metaphor

🇬🇧 ENGLISH

You don't learn to pilot a Boeing 747 on a live passenger flight; you practice in a flight simulator! Minikube and Kind (Kubernetes IN Docker) are zero-cost local simulators running right on your laptop where you can crash clusters safely. `kubectl` is the flight stick and control dashboard!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Kind (Kubernetes-in-Docker), her bir K8s düğümünü (Node) ayrı bir Docker konteyneri olarak ayağa kaldırır. Çok düğümlü (multi-node) yerel kümeleri saniyeler içinde başlatmak için idealdir. `kubectl`, `~/.kube/config` dosyasındaki kimlik doğrulama sertifikalarını ve küme uç noktalarını kullanarak doğrudan `kube-apiserver` ile HTTPS üzerinden REST haberleşmesi yapar. Temel bağlamlar (Context) arasında hızlı geçiş sağlar.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Kind runs containerized Kubernetes nodes using Docker containers as mock hardware nodes, making multi-node cluster provisioning instantaneous. `kubectl` parses `~/.kube/config` to authenticate and route encrypted HTTPS REST requests directly to the cluster's `kube-apiserver` endpoint.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# 1. Kind ile yerel bir K8s kümesi oluştur
kind create cluster --name eli5-cluster

# 2. Küme durumunu ve düğümleri listele
kubectl cluster-info
kubectl get nodes

# 3. Kubeconfig bağlamını kontrol et
kubectl config current-context

# 4. Tüm isim alanlarındaki pod'ları listele
kubectl get pods -A
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Çok fazla yerel küme açıp kapatırsanız Docker diskiniz ve belleğiniz şişebilir. Kullanmadığınız Kind kümelerini `kind delete cluster --name eli5-cluster` ile silmeyi unutmayın.

🇬🇧 WATCH OUT

Running multiple local Kind clusters consumes heavy RAM. Always delete unused clusters via `kind delete cluster` when done.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Kind ve Minikube sıfır maliyetle profesyonel K8s geliştirme ortamı sağlar.
- `kubectl` komut satırı aracı API Server ile konuşan anahtardır.
- `~/.kube/config` birden fazla kümenin erişim anahtarlarını barındırır.

- Kind and Minikube provide rapid, zero-cost local sandbox environments.
- `kubectl` translates CLI commands into structured JSON REST requests to API Server.
- The kubeconfig file manages tokens and contexts across multiple clusters.

12

BÖLÜM: KUBERNETES ILE TANIŞMA – ORKESTRA ŞEFI | MODULE: INTRODUCTION TO KUBERNETES – THE CONDUCTOR

İlk Spring Boot Pod Manifestomuzu Yazma (YAML Anatomisi)

Writing Your First Spring Boot Pod Manifest (YAML Anatomy)

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Kubernetes'e bir dilekçe yazıyoruz! Bu resmi dilekçenin 4 kutsal bölümü vardır: 1) Hangi kanun maddesi? (`apiVersion`), 2) Ne istiyorsun? (`kind: Pod`), 3) Bu şeyin adı ne olsun? (`metadata: name`), 4) Özellikleri neler olsun? (`spec: containers`). İşte bu 4 parçayı doğru yazarsanız K8s dilekçenizi saniyeler içinde işleme koyar!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Writing a Kubernetes YAML is filing an official petition to the cluster government! Every petition requires 4 sacred fields: 1) What law version? (`apiVersion`), 2) What resource type? (`kind: Pod`), 3) What is its name and tag? (`metadata`), 4) What are the exact specifications? (`spec`). Get those four right, and K8s grants your wish instantly!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Bir K8s manifestosunun yapısı: `apiVersion` (kaynağın API sürümü; örn: Pod için `v1`, Deployment için `apps/v1`), `kind` (oluşturulacak nesne tipi), `metadata` (name, namespace, labels, annotations gibi tanımlayıcı veriler), `spec` (kaynağın istenen teknik durumu; örn: imaj adı, portlar, ortam değişkenleri). `kubectl apply -f manifest.yaml` komutu verildiğinde API Server manifestoyu doğrular, etcd'ye kaydeder ve Kubelet konteyneri çeker.

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

The anatomy of every K8s resource: 1) `apiVersion` specifies the API endpoint group, 2) `kind` declares the schema type, 3) `metadata` houses names, namespaces, and queryable labels, 4) `spec` defines desired characteristics. Submitting via `kubectl apply -f` triggers schema validation in the API Server before etcd persistence.

☕ Java Spring Boot Pratiği ☕ Spring Boot in Action

```yaml
# spring-boot-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: spring-coffee-pod
  labels:
    app: coffee-service
    env: dev
spec:
  containers:
    - name: coffee-container
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      ports:
        - containerPort: 8080
          name: http
      resources:
        requests:
          memory: "256Mi"
          cpu: "100m"

# Pod'u çalıştır ve loglarını izle:
# kubectl apply -f spring-boot-pod.yaml
# kubectl logs -f spring-coffee-pod
```

⚠️ Acemi Tuzağı & Pro İpucu ⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

YAML dosyalarında TAB tuşu KESİNLİKLE yasaktır! Hizalama için sadece boşluk (space - genelde 2 boşluk) kullanılmalıdır, aksi takdirde YAML ayrıştırma hatası alırsınız.

🇬🇧 WATCH OUT

YAML syntax forbids TAB characters! Use strictly 2 spaces for indentation, or the API Server parser will reject the manifest with syntax errors.

🎯 Özet Çıkarımlar 🎯 Key Takeaways

- Her K8s kaynağı apiVersion, kind, metadata ve spec alanlarını içerir.
- Labels (etiketler) nesneleri gruplamak ve filtrelemek için hayati önem taşır.
- Pod logları `kubectl logs -f <pod-name>` ile anlık olarak takip edilir.

- Every K8s object schema shares apiVersion, kind, metadata, and spec.
- Labels are key-value pairs enabling dynamic grouping, querying, and routing.
- Inspect application stdout/stderr streams directly using `kubectl logs`.